

ບົດທີ 10: ການບໍາລຸງຮັກສາ ແລະ ວິວັດທະນາການຂອງຊອບແວ

Software Maintenance and Evolution



ການບໍາລຸງຮັກສາຊອບແວ: ຈຸດເລີ່ມຕົ້ນຂອງຂະບວນການຍາວນານ

ຫຼາຍຄົນຄິດວ່າເມື່ອ Deploy ຊອບແວເວີຊັນ 1.0 ໃຫ້ລູກຄ້າໃຊ້ງານ
ແລ້ວ, ໂຄງການກໍເປັນອັນສໍາເລັດ

ໃນໂລກຂອງວິສະວະກໍາຊອບແວ, ການ Deploy ຄື **"ຈຸດເລີ່ມຕົ້ນ"** ຂອງ
ໄລຍະທີ່ຍາວນານ ແລະ ມີຄ່າໃຊ້ຈ່າຍສູງທີ່ສຸດ

ນັ້ນກໍຄື **"ການບໍາລຸງຮັກສາ"** (Maintenance) ເຊິ່ງເປັນສ່ວນສໍາຄັນທີ່ສຸດ
ຂອງວົງຈອນຊີວິດຊອບແວ

"ການບໍາລຸງຮັກສາຊອບແວບໍ່ແມ່ນການແກ້ໄຂບັນຫາພຽງເທົ່ານັ້ນ, ແຕ່
ເປັນການວິວັດທະນາການຊອບແວໃຫ້ດີຂຶ້ນຕະຫຼອດເວລາ"



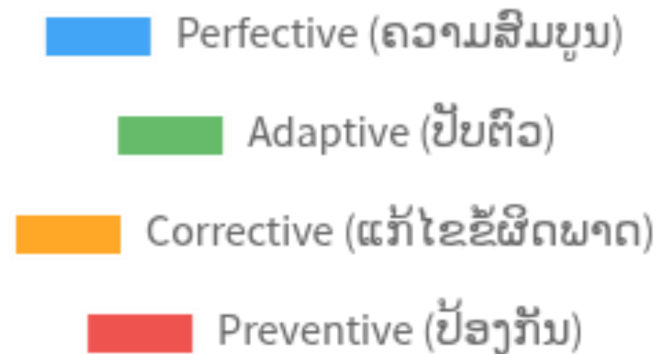
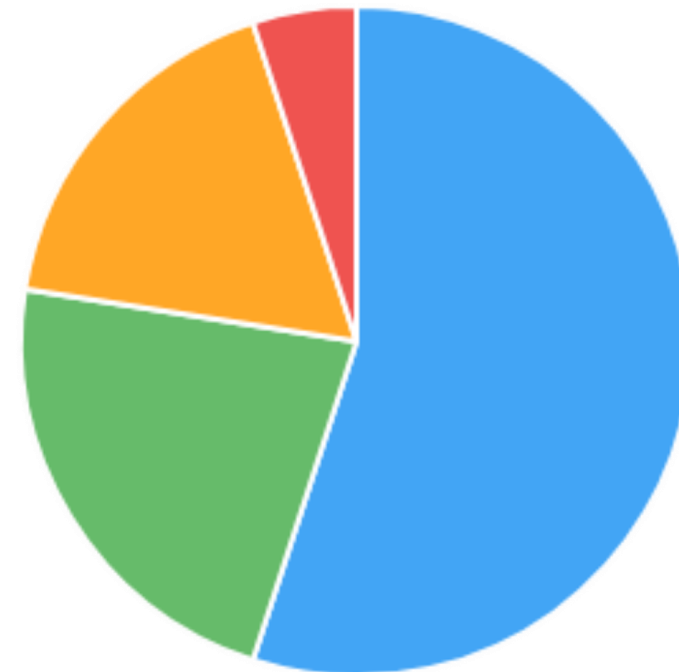
60-80%

ຂອງຄ່າໃຊ້ຈ່າຍທັງໝົດຂອງໂຄງການ

ແມ່ນຄ່າໃຊ້ຈ່າຍໃນການບໍາລຸງຮັກສາຊອບແວ

ປະເພດຂອງການບໍາລຸງຮັກສາ (4 ປະເພດ)

		Perfective Maintenance ການປັບປຸງເພື່ອຄວາມສົມບູນ	50-60%
		Adaptive Maintenance ການປັບຕົວ	20-25%
		Corrective Maintenance ການແກ້ໄຂຂໍ້ຜິດພາດ	15-20%
		Preventive Maintenance ການປ້ອງກັນ	5%



ການບໍາລຸງຮັກສາເພື່ອແກ້ໄຂຂໍ້ຜິດພາດ (Corrective Maintenance)

❓ ຄໍາຖາມຫຼັກ

"ມີຫຍັງເປັນເສຍຫາຍແດ່?"



ຄືກັບການ **"ສ້ອມແປງ"** ລົດ ເມື່ອມີບາງຢ່າງພັງ, ເຊັ່ນ: ການປ່ຽນຢາງລົດທີ່ຮົ່ວ ຫຼື ການສ້ອມແປງ ແອທີ່ບໍ່ເຢັນ.

ຕົວຢ່າງຈິງ

ຜູ້ໃຊ້ລາຍງານວ່າ ເມື່ອກົດປຸ່ມ "ສົ່ງອອກເປັນ PDF", ແອັບພລິເຄຊັນກໍ "crash". ທີມພັດທະນາກໍຈະເຂົ້າໄປ ສືບສວນ ແລະ ແກ້ໄຂໂຄດທີ່ເປັນສາເຫດ.

i ຄໍາອະທິບາຍ

ນີ້ຄືປະເພດທີ່ຄົນທົ່ວໄປຄິດເຖິງ. ມັນຄືການ **"ແກ້ໄຂ Bug"** ຫຼື ຂໍ້ຜິດພາດ (Faults) ທີ່ຖືກຄົ້ນພົບໂດຍຜູ້ໃຊ້ ຫຼັງຈາກ ທີ່ຊ້ອບແວໄດ້ຖືກປ່ອຍອອກໄປແລ້ວ.



15-20% ຂອງວຽກບໍາລຸງຮັກສາທັງໝົດ

ເຖິງແມ່ນຈະມີສັດສ່ວນບໍ່ຫຼາຍ, ແຕ່ການ ແກ້ໄຂຂໍ້ຜິດພາດມີຄວາມສໍາຄັນຕໍ່ການ ຮັກສາຄວາມເຊື່ອຖືຂອງຜູ້ໃຊ້

ການບໍາລຸງຮັກສາເພື່ອປັບຕົວ (Adaptive Maintenance)

❓ ຄໍາຖາມຫຼັກ

"ໂລກພາຍນອກປ່ຽນໄປແນວໃດ?"

⚙️ ຄືກັບການ **"ປັບປຸງ"** ລົດ ເພື່ອໃຫ້ສາມາດໃຊ້ກັບນໍ້າມັນປະເພດໃໝ່ໄດ້ ຫຼື ເພື່ອໃຫ້ຜ່ານມາດຕະຖານຄວັນພິດໃໝ່ຂອງລັດຖະບານ.

🔄 ຕົວຢ່າງຈິງ

ລະບົບປະຕິບັດການມີການອັບເດດເວີຊັນໃໝ່ (ເຊັ່ນ: Windows 11, Android 14), ເຮັດໃຫ້ຊັອບແວຂອງເຮົາບາງສ່ວນເຮັດວຽກບໍ່ໄດ້. ເຮົາຈຶ່ງຕ້ອງແກ້ໄຂໂຄດເພື່ອໃຫ້ຮອງຮັບ OS ໃໝ່.

ℹ️ ຄໍາອະທິບາຍ

ຊັອບແວບໍ່ໄດ້ຢູ່ຢ່າງໂດດດ່ຽວ. ມັນຕ້ອງເຮັດວຽກໃນ **"ສະພາບແວດລ້ອມ"** ທີ່ປ່ຽນແປງຢູ່ຕະຫຼອດເວລາ. Adaptive Maintenance ຄືການ **"ດັດແກ້"** ຊັອບແວ ເພື່ອໃຫ້ມັນຍັງຄົງສາມາດເຮັດວຽກໄດ້ ເມື່ອສະພາບແວດລ້ອມຂອງມັນປ່ຽນໄປ.



20-25% ຂອງວຽກບໍາລຸງຮັກສາທັງໝົດ

ການປັບຕົວຊັອບແວໃຫ້ເຂົ້າກັບສະພາບແວດລ້ອມທີ່ປ່ຽນແປງ ແມ່ນສິ່ງຈໍາເປັນສໍາລັບການຮັກສາຄວາມສາມາດໃນການໃຊ້ງານ



ປ່ຽນ OS



ປ່ຽນຖານຂໍ້ມູນ



ປ່ຽນກົດໝາຍ

ການບໍາລຸງຮັກສາເພື່ອຄວາມສົມບູນ (Perfective Maintenance)

❓ ຄໍາຖາມຫຼັກ

"ເຮົາຈະເຮັດໃຫ້ມັນດີຂຶ້ນໄດ້ແນວໃດ?"

↑ ຄືກັບການ **"ອັບເກຣດ"** ລົດ. ເຈົ້າຂອງລົດອາດຈະຕ້ອງການເພີ່ມລະບົບ GPS, ປ່ຽນເຄື່ອງສຽງໃໝ່, ຫຼື ຕິດກ້ອງຫຼັງ.

+ ຕົວຢ່າງຈິງ

ລູກຄ້າຂໍໃຫ້ເພີ່ມຄວາມສາມາດໃນການຈ່າຍເງິນຜ່ານ QR Code ເຂົ້າໄປໃນແອັບຢູ ຮ້ານກາເຟ. ຜູ້ໃຊ້ຮ້ອງຂໍໃຫ້ປັບປຸງໜ້າຈໍຄົ້ນຫາ ໃຫ້ສາມາດກັ່ນຕອງ (Filter) ຕາມປະເພດສິນຄ້າໄດ້.

i ຄໍາອະທິບາຍ

ນີ້ຄືປະເພດທີ່ກວມເອົາວຽກສ່ວນໃຫຍ່ທີ່ສຸດ. ມັນຄືການ **"ເພີ່ມ Feature ໃໝ່"** ຫຼື **"ປັບປຸງ Feature ເດີມ"** ຕາມຄວາມຕ້ອງການໃໝ່ໆ ຂອງລູກຄ້າ ຫຼື ຜູ້ໃຊ້. ມັນເປັນການເຮັດໃຫ້ຊັອບແວ "ສົມບູນແບບ" ຫຼາຍຂຶ້ນ.



50-60% ຂອງວຽກບໍາລຸງຮັກສາທັງໝົດ

ການເພີ່ມ ແລະ ປັບປຸງຄວາມສາມາດແມ່ນສ່ວນທີ່ສໍາຄັນທີ່ສຸດໃນການຮັກສາຄວາມພໍໃຈຂອງຜູ້ໃຊ້ ແລະ ການແຂ່ງຂັນ



ເພີ່ມຄວາມ
ສາມາດໃໝ່



ປັບປຸງ
ປະສິດທິພາບ



ປັບປຸງປະສິບ
ການຜູ້ໃຊ້

ການບໍາລຸງຮັກສາເພື່ອປ້ອງກັນ (Preventive Maintenance)

❓ ຄໍາຖາມຫຼັກ

"ເຮົາຈະປ້ອງກັນບໍ່ໃຫ້ບັນຫາເກີດຂຶ້ນໃນອະນາຄົດໄດ້ແນວໃດ?"



ຄືກັບການ **"ກວດເຊັກສະພາບລົດ"** ຕາມກຳນົດ. ເຮົາປ່ຽນນ້ຳມັນເຄື່ອງ, ກວດສອບລະບົບເບຣກ, ບໍ່ແມ່ນເພາະມັນພັງ, ແຕ່ເພື່ອ "ປ້ອງກັນ" ບໍ່ໃຫ້ມັນພັງໃນລະຫວ່າງການເດີນທາງ.

<> ຕົວຢ່າງຈິງ

ການ Refactor ໂຄດ: ປັບປຸງໂຄງສ້າງຂອງໂຄດທີ່ຊັບຊ້ອນ ແລະ ອ່ານຍາກ ໃຫ້ສະອາດ ແລະ ເຂົ້າໃຈງ່າຍຂຶ້ນ. ການອັບເດດ Libraries ໃຫ້ເປັນເວີຊັນລ່າສຸດ ເພື່ອປົດຈຸດອ່ອນ/ຂໍ້ຜິດພາດດ້ານຄວາມປອດໄພ.

i ຄໍາອະທິບາຍ

ຄືການດັດແກ້ຊອບແວໃນປັດຈຸບັນ ເພື່ອເຮັດໃຫ້ມັນ **"ບໍາລຸງຮັກສາງ່າຍຂຶ້ນ"** ໃນອະນາຄົດ ແລະ **"ຫຼຸດໂອກາດທີ່ຈະເກີດ Bug"**. ມັນຄືການ "ຈ່າຍໜ້ອຍທາງເທັກນິກ" (Paying back Technical Debt).



5% ຂອງວຽກບໍາລຸງຮັກສາທັງໝົດ

ເຖິງແມ່ນຈະມີສັດສ່ວນນ້ອຍ, ແຕ່ການປ້ອງກັນມີຄວາມສໍາຄັນຕໍ່ການຫຼຸດຜ່ອນຄ່າໃຊ້ຈ່າຍໃນອະນາຄົດ ແລະ ການຮັກສາຄຸນນະພາບຊື່ອບແວ



Refactor ໂຄດ



ອັບເດດ Libraries



ປັບປຸງເອກະສານ

ການຈັດການໜີ້ທາງເທັກນິກ: ຄໍານິຍາມ ແລະ ສາເຫດທີ່ເກີດ

☰ ຄໍານິຍາມ

Technical Debt ແມ່ນ "ຜົນສະທ້ອນໃນອະນາຄົດ ຂອງ ການເລືອກທາງລັດ ຫຼື ການອອກແບບທີ່ບໍ່ດີໃນປັດຈຸບັນ"

ແນວຄິດນີ້ຖືກສ້າງຂຶ້ນໂດຍ Ward Cunningham (ໜຶ່ງໃນ ຜູ້ສ້າງ Agile Manifesto)

➡ ການປຽບທຽບກັບໜີ້ສິນທາງການເງິນ

ການສ້າງໜີ້: ການເລືອກຂຽນໂຄດແບບ "ທາງລັດ" ເພື່ອ ໃຫ້ Feature ສໍາເລັດທັນກໍານົດ

ການຈ່າຍດອກເບ້ຍ: ໃນອະນາຄົດ, ທຸກໆຄັ້ງທີ່ຕ້ອງກັບ ມາແກ້ໄຂ ຫຼື ຕໍ່ເຕີມໂຄດ, ມັນຈະ "ຍາກຂຶ້ນ ແລະ ຊ້າລົງ"



ສາເຫດທີ່ເກີດ Technical Debt

ເຂົ້າໃຈສາເຫດທີ່ເຮັດໃຫ້ເກີດໜີ້ທາງເທັກນິກແມ່ນຂັ້ນຕອນ ທໍາອິດໃນການປ້ອງກັນ ແລະ ຈັດການມັນ



ຄວາມກົດດັນທາງ
ທຸລະກິດ



ການຂາດຄວາມ
ເຂົ້າໃຈ



ໂຄດທີ່ຊັບຊ້ອນເກີນ
ໄປ



ການຂາດການທົດ
ສອບ

ໜີ້ທາງເທັກນິກ: ປະເພດ ແລະ ຕົວຢ່າງ

▣ Technical Debt Quadrant

		Reckless & Deliberate	
		"We don't have time for design!"	
		Prudent & Deliberate	
		"We must ship now and deal with consequences later"	
Reckless	Inadvertent	Prudent & Inadvertent	
	Reckless & Inadvertent	"Now we know how we should have done it"	
		"What's Layering?"	

≡ ຕົວຢ່າງຂອງໜີ້ທາງເທັກນິກ



ໂຄດທີ່ຊ້ຳຊ້ອນ
(Duplicated Code)



ການຕັ້ງຊື່ທີ່ບໍ່ດີ (Bad Naming)



ການຂາດ Unit Test



ການ Hardcode ຄ່າຄົງທີ່

⚠ ຜົນກະທົບຂອງໜີ້ທາງເທັກນິກ

🕒 ຜົນກະທົບຕໍ່ເວລາ

- ການແກ້ໄຂບັນຫາ ຊ້າລົງ
- ການເພີ່ມ Feature ໃໝ່ ຍາກຂຶ້ນ
- ຄວາມສາມາດໃນການຂຽນໂຄດ ຫຼຸດລົງ

💰 ຜົນກະທົບຕໍ່ຄ່າໃຊ້ຈ່າຍ

- ຄ່າໃຊ້ຈ່າຍໃນການບໍາລຸງຮັກສາ ເພີ່ມຂຶ້ນ
- ລະບົບຕ້ອງການທັງໝົດ ສູງຂຶ້ນ
- ການຝຶກອົບຮົມທີມງານໃໝ່ ຍາກຂຶ້ນ

ການຈັດການໜີ້ທາງເທກນິກ: ການເຮັດໃຫ້ເຫັນໄດ້ ແລະ ການຈ່າຍຄືນ

🔍 ເຮັດໃຫ້ໜີ້ສາມາດເບິ່ງເຫັນໄດ້ (Make Debt Visible)

ສ້າງ "ວຽກ" (Task/Story) ໃນ Product Backlog ໂດຍສະເພາະ, ພ້ອມທັງໃສ່ Tag ວ່າ Technical Debt

🔄 ຈ່າຍຄືນຢ່າງສະໝໍ່າສະເໝີ (Pay it back regularly)

ຈັດສັນເວລາສ່ວນໜຶ່ງໃນທຸກໆ Sprint ເພື່ອ "ຈ່າຍຄືນໜີ້"

⚙️ ໃຊ້ Automation ເປັນເຄື່ອງມື

ການມີ CI Pipeline ທີ່ລັ້ນ Unit Test ທີ່ຄົບຖ້ວນ ຈະສ້າງ "ຕາໜ່າງຄວາມປອດໄພ" ທີ່ຊ່ວຍໃຫ້ທີມງານກ້າ Refactor

"ຈົ່ງປະໃຫ້ແຄ້ມສະອາດກວ່າຕອນທີ່ເຈົ້າມາຮອດ."

— ກົດຂອງ Boy Scout



ການຈັດການໜີ້ທາງເທກນິກ

ທີມທີ່ດີບໍ່ແມ່ນທີມທີ່ບໍ່ມີໜີ້, ແຕ່ເປັນທີມທີ່ "ຮູ້ວ່າຕົນເອງ ມີໜີ້ເທົ່າໃດ, ຢູ່ໃສ ແລະ ມີແຜນການທີ່ຈະຈ່າຍຄືນຢ່າງ ສະໝໍ່າສະເໝີ"



ຈັດລາຍການໜີ້



ກຳນົດເວລາຈ່າຍຄືນ



ຊໍາລະບົບ

ການ Refactoring ແລະ Reengineering: ຄວາມແຕກຕ່າງ

ການຈັດລະບຽບ

Refactoring

-  ປັບປຸງໂຄງສ້າງພາຍໃນ ໂດຍບໍ່ປ່ຽນແປງພຶດຕິກຳພາຍນອກ
-  ຂະໜາດນ້ອຍ, ເຮັດຕະຫຼອດເວລາ
-  ເຮັດໃຫ້ໂຄດສະອາດ, ອ່ານງ່າຍ, ເຂົ້າໃຈງ່າຍຂຶ້ນ
-  ເປັນສ່ວນໜຶ່ງຂອງວົງຈອນ Red-Green-Refactor ໃນ TDD

❏ ຕົວຢ່າງ

Extract Method, Rename Variable,
Replace Magic Number with Constant

Reengineering

-  ກວດສອບ ແລະ ປ່ຽນແປງລະບົບເກົ່າ ເພື່ອສ້າງມັນຂຶ້ນມາໃໝ່
-  ອາດຈະປ່ຽນສະຖາປັດຕະຍະກຳຂອງລະບົບ
-  ຂະໜາດໃຫຍ່, ເຮັດເປັນໂຄງການ
-  ປັບປຸງຄຸນນະພາບ, ເພີ່ມປະສິດທິພາບ, ຫຼື ຍ້າຍລະບົບໄປສູ່ເທັກໂນໂລຢີໃໝ່

❖ ຂັ້ນຕອນຂອງ Reengineering

Reverse Engineering → Restructuring
→ Forward Engineering

ການຈັດການລະບົບເກົ່າ: ຄຳນິຍາມ ແລະ ຄວາມສຳຄັນ

🕒 ຄຳນິຍາມ

Legacy System ແມ່ນ **ລະບົບຊອບແວເກົ່າ** ທີ່ຍັງຄົງມີຄວາມສຳຄັນຕໍ່ການດຳເນີນທຸລະກິດ, ແຕ່ມີຄວາມຍາກລຳບາກໃນການບຳລຸງຮັກສາ ແລະ ປັບປຸງ

⚠️ ລັກສະນະທີ່ມັກຈະພົບເຫັນ

🖥️ ເທັກໂນໂລຢີທີ່ຫຼ້າສະໄໝ

📄 ຂາດເອກະສານ

🎓 ຂາດຄວາມຮູ້

👤 ໂຄງສ້າງທີ່ຊັບຊ້ອນ

🔧 ຂາດການທົດສອບອັດຕະໂນມັດ

The Legacy System Dilemma



ຄວາມສ່ຽງ ແລະ ຄ່າໃຊ້ຈ່າຍໃນການປ່ຽນແປງ



ຄຸນຄ່າທາງທຸລະກິດທີ່ຍັງມີຢູ່

❓ ເປັນຫຍັງບໍ່ "ຖິ້ມ" ມັນໄປ?

- ✅ **ມັນຍັງເຮັດວຽກໄດ້** - ເປັນ "ຫົວໃຈ" ຂອງທຸລະກິດມາເປັນເວລາຫຼາຍສິບປີ
- 💰 **ຄ່າໃຊ້ຈ່າຍໃນການສ້າງໃໝ່ສູງເກີນໄປ** - ອາດຈະຕ້ອງໃຊ້ງົບປະມານຫຼາຍຮ້ອຍລ້ານໂດລາ
- 🛡️ **ຄວາມສ່ຽງສູງ** - ການປ່ຽນຖ່າຍຈາກລະບົບເກົ່າໄປສູ່ລະບົບໃໝ່ ເປັນຂະບວນການທີ່ມີຄວາມສ່ຽງສູງ

ທາງເລືອກໃນການຈັດການ Legacy Systems (The 5 R's)

5 ບຸດທະສາດຫຼັກໃນການຈັດການລະບົບເກົ່າ

Retain (ຮັກສາໄວ້ຄືເກົ່າ)

ບໍ່ເຮັດຫຍັງເລີຍ. ຍອມຮັບລະບົບຕາມທີ່ມັນເປັນ ແລະ ສືບຕໍ່
ບຳລຸງຮັກສາແບບ Corrective (ແກ້ Bug) ຕໍ່ໄປ.

ເມື່ອ: ລະບົບຍັງຄົງໝັ້ນຄົງ, ບໍ່ຄ່ອຍມີການປ່ຽນແປງ

Retire (ປິດກະສຽນ)

ຍົກເລີກການໃຊ້ງານລະບົບນັ້ນໂດຍສິ້ນເຊີງ.

ເມື່ອ: ໜ້າທີ່ການໃຊ້ງານຂອງລະບົບນັ້ນ ບໍ່ມີຄວາມຈຳເປັນຕໍ່
ທຸລະກິດອີກ

Rehost (ຍ້າຍບ່ອນຢູ່)

ຍ້າຍລະບົບຈາກສະພາບແວດລ້ອມເກົ່າ (ເຊັ່ນ: Mainframe
Server) ໄປສູ່ສະພາບແວດລ້ອມໃໝ່ (ເຊັ່ນ: Cloud
Infrastructure) ໂດຍບໍ່ມີການປ່ຽນແປງໂຄດ.

ເມື່ອ: ເປົ້າໝາຍຫຼັກຄືການຫຼຸດຕົ້ນທຶນດ້ານ Infrastructure

Replatform (ປັບປ່ຽນແພລດຟອມ)

ຄ້າຍຄືກັບ Rehost, ແຕ່ມີການ "ປັບປ່ຽນເລັກນ້ອຍໆ" ເພື່ອ
ໃຫ້ສາມາດໃຊ້ປະໂຫຍດຈາກຄວາມສາມາດຂອງແພລດຟອມໃ
ໝ່ໄດ້.

ເມື່ອ: ຕ້ອງການປັບປ່ຽນບາງສ່ວນເພື່ອຮັບປະໂຫຍດຈາກ
ແພລດຟອມໃໝ່

ທາງເລືອກໃນການຈັດການ Legacy Systems (The 5 R's)

5 ຍຸດທະສາດຫຼັກໃນການຈັດການລະບົບເກົ່າ

Re-architect / Rebuild / Replace (ສ້າງໃໝ່)

ນີ້ຄືທາງເລືອກທີ່ໃຫຍ່ທີ່ສຸດ. ມັນຄືການ Reengineering ຫຼື ການສ້າງລະບົບຂຶ້ນມາໃໝ່ທັງໝົດ. ມີຫຼາຍຍຸດທະສາດໃນການເຮັດ, ເຊັ່ນ: Big Bang ຫຼື Strangler Fig Pattern.

ເມື່ອ: ລະບົບເກົ່າມີ "ໜີ້ທາງເທັກນິກ" ສະສົມຫຼາຍຈົນເກີນໄປ

 ການເລືອກຍຸດທະສາດທີ່ເໝາະສົມຂຶ້ນກັບສະຖານະການຂອງລະບົບ, ງົບປະມານ, ແລະຄວາມສ່ຽງທີ່ອົງກອນສາມາດຮັບຜິດຊອບໄດ້

Strangler Fig Pattern: ການຄ່ອຍໆ ແທນທີ່ລະບົບເກົ່າ

📍 Strangler Fig Pattern

ເປັນວິທີທີ່ນິຍົມໃນປັດຈຸບັນ ເຊິ່ງຄືການ ຄ່ອຍໆ ສ້າງລະບົບໃໝ່ຂຶ້ນມາ "ອ້ອມຮອບ" ລະບົບເກົ່າ

🔄 ການດຳເນີນງານ

ເມື່ອ Feature ໃໝ່ສ້າງສໍາເລັດ, ກໍຈະຄ່ອຍໆ ປ່ຽນເສັ້ນທາງການເອິ້ນໃຊ້ ຈາກລະບົບເກົ່າ ມາຫາລະບົບໃໝ່ທີ່ລະສ່ວນ, ຈົນໃນທີ່ສຸດລະບົບເກົ່າກໍຈະ "ຖືກຮັບ" ຈົນບໍ່ມີໃຜໃຊ້ ແລະ ສາມາດປິດຖິ້ມໄດ້

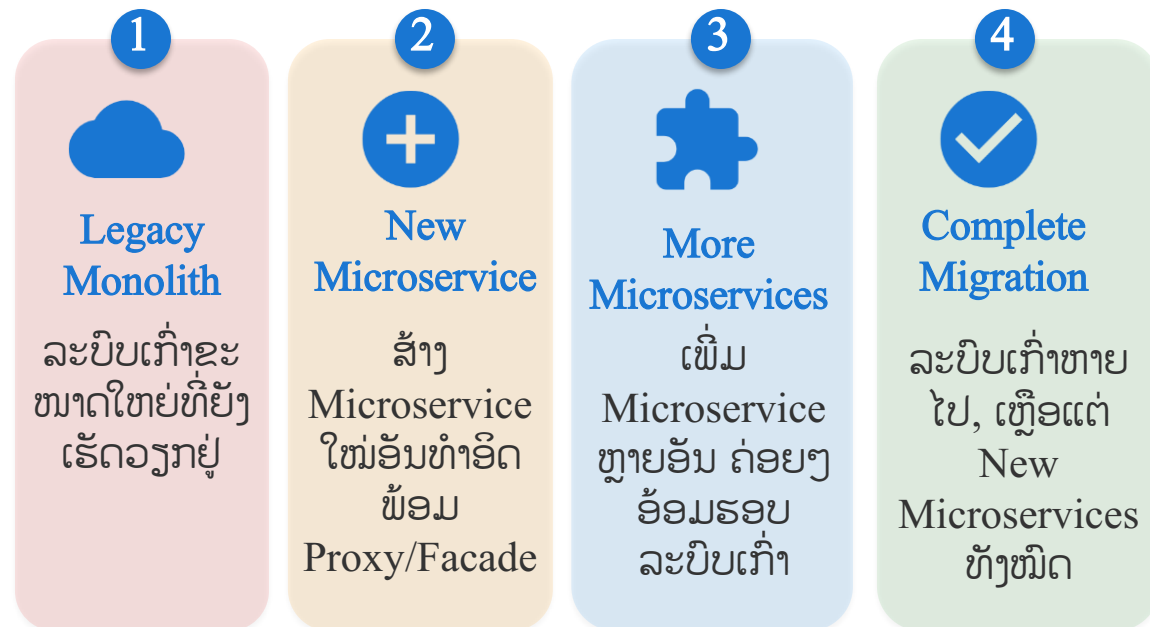
👍 ປະໂຫຍດຂອງ Strangler Fig Pattern

🛡️ ຄວາມສ່ຽງຕໍ່າ

🔄 ການປ່ຽນແປງທີ່ຕໍ່ເນື່ອງ

🏠 ການຈັດສັນຄ່າໃຊ້ຈ່າຍ

🛠️ ທີ່ມງານສາມາດຮຽນຮູ້ໄດ້



➡️ ການປຽບທຽບກັບວິທີອື່ນ

Big Bang

ປ່ຽນທັງໝົດໃນເວລາດຽວ
ຄວາມສ່ຽງ: ສູງ

Strangler Fig

ຄ່ອຍໆ ປ່ຽນແປງທີ່ລະສ່ວນ
ຄວາມສ່ຽງ: ຕໍ່າ

ການບໍາລຸງຮັກສາ ແລະ ວິວັດທະນາການຊອບແວແມ່ນສິ່ງຈໍາເປັນ

- 1 ການເຂົ້າໃຈປະເພດຂອງການບໍາລຸງຮັກສາຊ່ວຍໃຫ້ເຮົາເຫັນພາບວ່າວຽກຂອງວິສະວະກອນຊອບແວ **ບໍ່ໄດ້ຈົບລົງ** ຫຼັງຈາກການ Deploy
- 2 ການຈັດການໜີ້ທາງເທັກນິກບໍ່ແມ່ນການລະເລີຍ, ແຕ່ຕ້ອງຈັດການ **ຢ່າງມີສະຕິ**
- 3 ການ Refactoring ແລະ Reengineering ເປັນສອງກິດຈະກຳທີ່ສໍາຄັນໃນການຈັດການວົງຈອນຊີວິດຂອງຊອບແວ
- 4 ການຈັດການ Legacy Systems ບໍ່ມີຄໍາຕອບທີ່ຕາຍຕົວ, ຕ້ອງພິຈາລະນາສະພາບການຂອງແຕ່ລະລະບົບ
- 5 ການຂຽນໂຄດທີ່ສະອາດ ແລະ ມີສະຖາປັດຕະຍະກຳທີ່ດີ **ເປັນສິ່ງທີ່ສໍາຄັນທີ່ສຸດ**



ການບໍາລຸງຮັກສາ ແລະ ວິວັດທະນາການຊອບແວ
ແມ່ນການລົງທຶນໃນອະນາຄົດ

"ວິສະວະກອນຊອບແວທີ່ດີ ບໍ່ແມ່ນບໍ່ມີໜີ້, ແຕ່ໃຫ້ຮູ້ວ່າຕົນເອງມີໜີ້ເທົ່າໃດ, ຢູ່ໃສ ແລະ ມີແຜນການທີ່ຈະຈ່າຍຄືນຢ່າງສະໝໍ່າສະເໝີ"